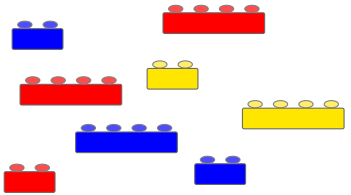
Clustering with K-means

Udit Bhatia

ES 335 / ES 678 Machine Learning



Learning without labels



Sort by colour? by size?

Supervised

Given $(\mathbf{x}_i, y_i)$. Learn $f(\mathbf{x}) \approx y$.

Unsupervised

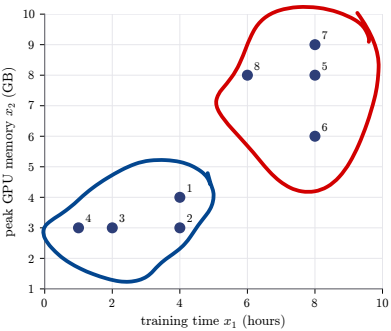
Given only $\mathbf{x}_i$. Find which points belong together.

No y, so no error to measure.

Where clustering is used

- Customer segments
- Image compression: K colours
- Grouping documents and images by their embeddings
- Vector search: compare a query with cluster centres first
- Scheduling training runs by resource use

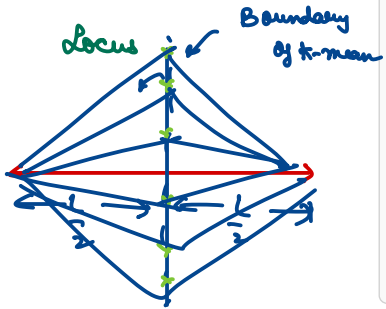
Eight training runs, no labels



run i	1	2	3	4	5	6	7	8
x_1 (hours)	4	4	2	1	8	8	8	6
x_2 (GB)	4	3	3	3	8	6	9	8

Two groups by eye.

With 10 000 runs and 50 features?



The K-means objective

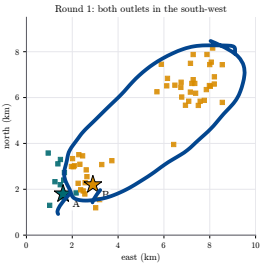
Choose labels $g_i \in \{1, \ldots, K\}$ and centroids $\boldsymbol{\mu}_k$ to minimise

$$J = \sum_{i=1}^{n} \|\mathbf{x}_i - \boldsymbol{\mu}_{g_i}\|^2$$

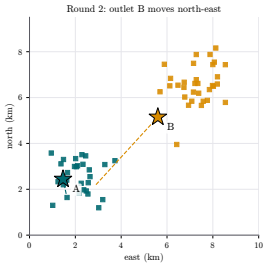
$\frac{\partial J}{\partial \mu} = 0$

- Two unknowns: labels and centroids
- All labellings: K^n (2^{1000} has 302 digits)
- Fix one unknown $\Rightarrow$ the other is easy

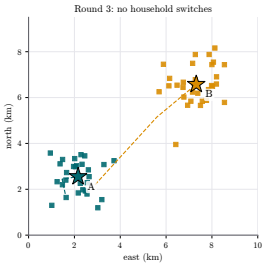
The idea: two pizza outlets



Round 1



Round 2



Round 3: stop

Summary

Order from the nearest outlet. **Move** each outlet to the average of its customers.

Lloyd's algorithm

Algorithm 1

Input: points $\mathbf{x}_1, \ldots, \mathbf{x}_n$; K; initial centroids



1: **repeat**
2: $\quad g_i \leftarrow \arg\min_k \|\mathbf{x}_i - \boldsymbol{\mu}_k\|^2$ for every i — assign
3: $\quad \boldsymbol{\mu}_k \leftarrow$ mean of cluster k, for every k — update
4: **until** no label changes

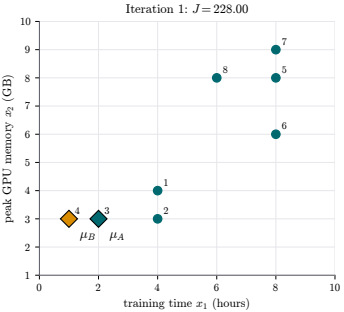
Start (deliberately bad): $\mu_A = (2, 3)$, $\mu_B = (1, 3)$.

Iteration 1: assign

run	$\mathbf{x}_i$	$\|\mathbf{x}_i - \mu_A\|^2$ $\mu_A = (2.000, 3.000)$	$\|\mathbf{x}_i - \mu_B\|^2$ $\mu_B = (1.000, 3.000)$	assigned to
1	$(4, 4)$	**5.00**	10.00	A
2	$(4, 3)$	**4.00**	9.00	A
3	$(2, 3)$	**0.00**	1.00	A
4	$(1, 3)$	1.00	**0.00**	B
5	$(8, 8)$	**61.00**	74.00	A
6	$(8, 6)$	**45.00**	58.00	A
7	$(8, 9)$	**72.00**	85.00	A
8	$(6, 8)$	**41.00**	50.00	A

Run 1: 5 to μ_A, 10 to $\mu_B \Rightarrow$ A

$J = 228.00$



Iteration 1: update

Cluster A = runs 1, 2, 3, 5, 6, 7, 8:

$$\mu_A = \left(\frac{40}{7},\ \frac{41}{7}\right) = (5.714, 5.857)$$

Cluster B = run 4 alone: $\mu_B = (1, 3)$.

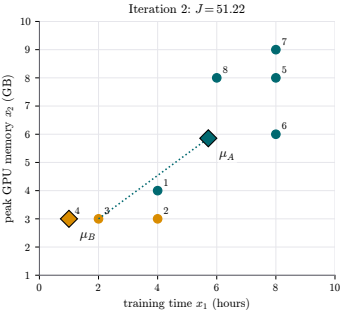
Insight

μ_A is dragged towards the large runs.

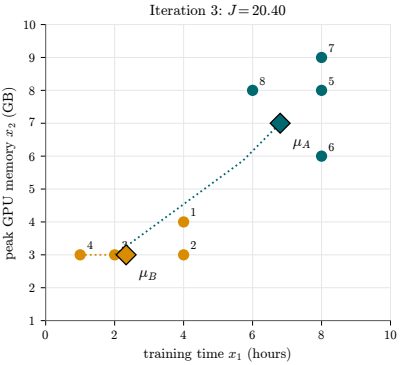
Iteration 2: runs 2 and 3 move to B

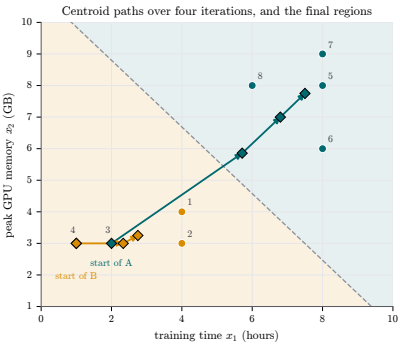
run	$\mathbf{x}_i$	$\|\mathbf{x}_i - \mu_A\|^2$ $\mu_A = (5.714, 5.857)$	$\|\mathbf{x}_i - \mu_B\|^2$ $\mu_B = (1.000, 3.000)$	assigned to
1	(4, 4)	**6.39**	10.00	A
2	(4, 3)	11.10	**9.00**	B
3	(2, 3)	21.96	**1.00**	B
4	(1, 3)	30.39	**0.00**	B
5	(8, 8)	**9.82**	74.00	A
6	(8, 6)	**5.24**	58.00	A
7	(8, 9)	**15.10**	85.00	A
8	(6, 8)	**4.67**	50.00	A

J: 228.00 $\rightarrow$ 51.22



Iterations 3 and 4: stop





J: 228.00 $\rightarrow$ 51.22 $\rightarrow$ 20.40 $\rightarrow$ 15.25 Best of all 256.

Update step: the best centroid is the mean

One cluster, one coordinate, values $x_1, \ldots, x_m$:

$$f(c) = \sum_{j=1}^{m} (x_j - c)^2$$

$$f'(c) = -2 \sum_{j=1}^{m} (x_j - c) = 0 \implies \boxed{c = \frac{1}{m} \sum_{j=1}^{m} x_j}$$

$f''(c) = 2m > 0$: a minimum.

Small runs, first coordinate $4, 4, 2, 1$ (mean 2.75):

$f(2) = 9.00, \quad f(2.75) = 6.75, \quad f(3.5) = 9.00.$

Why squared distance?

- Squared $\Rightarrow$ mean (cheap)
- Absolute $\Rightarrow$ median (K-medians)

Warning

Outliers pull the mean: $1, 2, 3, 100$ has mean 26.5, median 2.5.

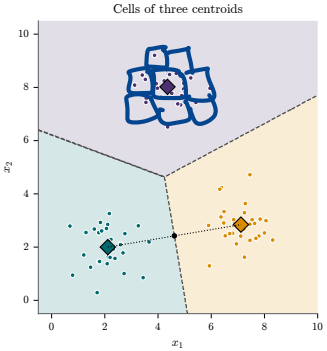
Assignment step: the boundary is a straight line

$\mathbf{x}$ is nearer $\boldsymbol{\mu}_a$ than $\boldsymbol{\mu}_b$ when

$$2(\boldsymbol{\mu}_b - \boldsymbol{\mu}_a)^\top \mathbf{x} < \|\boldsymbol{\mu}_b\|^2 - \|\boldsymbol{\mu}_a\|^2$$

Linear in $\mathbf{x}$: the perpendicular bisector.

Eight runs: $9.5\,x_1 + 9\,x_2 = 98.1875$



J never increases, so the algorithm stops

- **Assign:** nearest centroid $\Rightarrow$ J cannot rise
- **Update:** mean is best $\Rightarrow$ J cannot rise
- Finitely many labellings $\Rightarrow$ it stops

Warning

Stopping does not mean best.

Numbers $0, 1, 2, 4$ with $K = 2$:

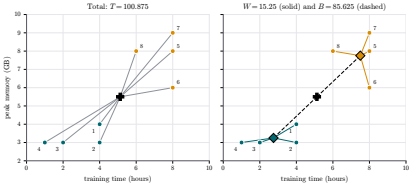
it can stop at $\{0,1\}\ \{2,4\}$: $J = 0.25 + 0.25 + 1 + 1 = 2.5$

yet better is $\{0,1,2\}\ \{4\}$: $J = 1 + 0 + 1 + 0 = 2$

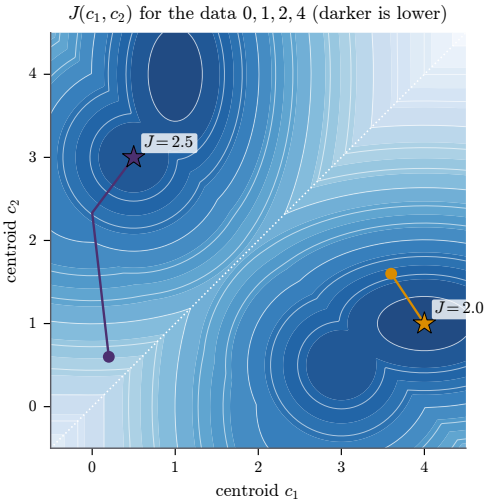
Within-cluster and between-cluster spread: $T = W + B$

$$\underbrace{\sum_i \|\mathbf{x}_i - \bar{\mathbf{x}}\|^2}_{T} = \underbrace{\sum_i \|\mathbf{x}_i - \boldsymbol{\mu}_{g_i}\|^2}_{W=J} + \underbrace{\sum_k n_k \|\boldsymbol{\mu}_k - \bar{\mathbf{x}}\|^2}_{B}$$

- W: spread inside clusters
- B: spread between cluster means
- T fixed $\Rightarrow$ smaller W = larger B
- Eight runs: $15.25 + 85.625 = 100.875$



J has several valleys



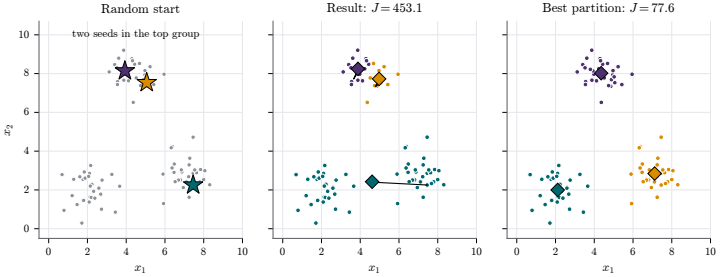
Data $0, 1, 2, 4$, $K = 2$.

Stars: the two stopping points, $J = 2.5$ and $J = 2$.

Local minimum: no step lowers J, yet a better answer exists.

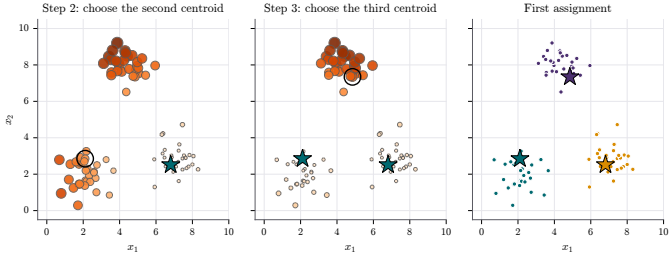
Best answer: NP-hard to find.

A random start can fail



- Two centroids in one group: 77% of random starts
- Result $J = 453.1$ vs best 77.6

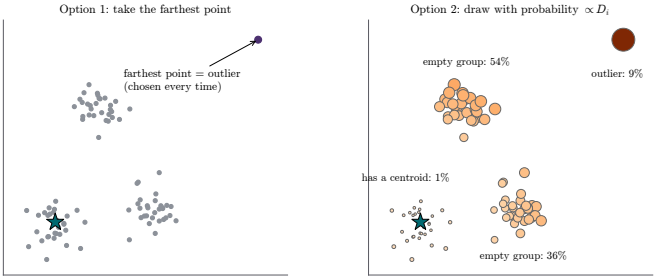
The fix: spread the starting centroids out



1. First centroid: a random data point (star).
2. D_i = squared distance from point i to its nearest chosen centroid.
3. Next centroid: draw a point with probability $\propto D_i$ (bigger disc = likelier). Repeat.

Why not simply take the farthest point?

Goal: next centroid in an empty group. Star = first centroid.



- Farthest point: always the outlier ⇒ centroid wasted.
- Drawing: 30 far points outweigh 1 ⇒ 90% in an empty group.

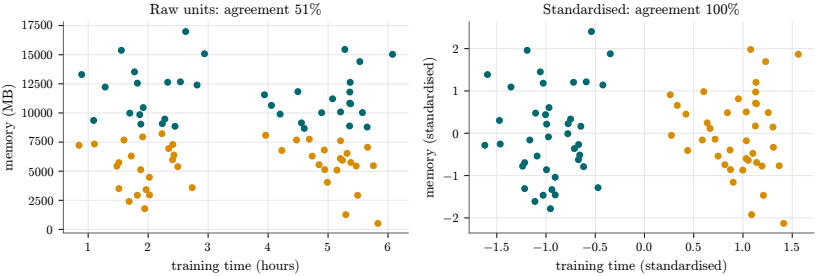
K-means++

Algorithm 2: K-means++ initialisation

1: $\boldsymbol{\mu}_1 \leftarrow$ a data point chosen at random
2: **for** $k = 2, \ldots, K$ **do**
3: $\quad D_i \leftarrow$ squared distance from $\mathbf{x}_i$ to its nearest chosen centroid
4: $\quad \boldsymbol{\mu}_k \leftarrow \mathbf{x}_i$ with probability $D_i / \sum_l D_l$
5: **end for**
6: run Lloyd's algorithm from $\boldsymbol{\mu}_1, \ldots, \boldsymbol{\mu}_K$

90 points, 400 starts each: best partition reached random 84% vs K-means++ 97.8%

Standardise the features first



Hours vs megabytes: agreement 51% $\rightarrow$ 100% after standardising

How many clusters, K?

K = number of clusters. We choose it before running K-means.

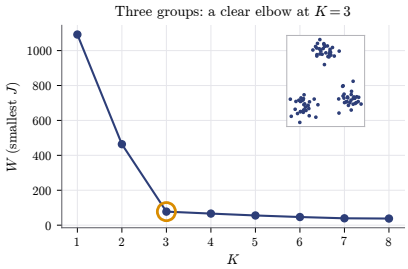
Smallest J for the eight runs:

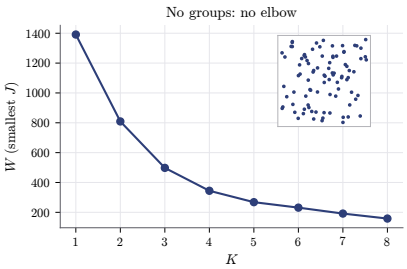
K	1	2	3	4	5	6	7	8
best J	100.875	15.250	8.750	4.333	1.500	1.000	0.500	0.000

More clusters $\Rightarrow$ smaller J, down to $J = 0$ at $K = n$. So J alone cannot choose K.

1. Elbow method

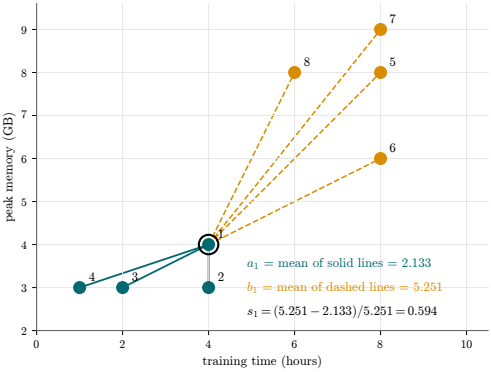
W_K = smallest J with K clusters. **Elbow:** the K after which W_K stops falling fast.





Drops: 628.1, 386.3, 11.2 $\Rightarrow$ bend at $K = 3$. Weakness: often no clear bend (right).

2. Silhouette of one point



a_i = mean distance to the other points of its own cluster

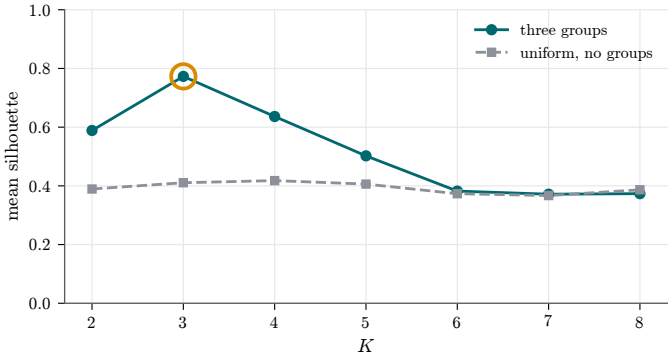
b_i = mean distance to the points of the nearest other cluster

$$s_i = \frac{b_i - a_i}{\max(a_i, b_i)}$$

$s_i \approx 1$: well placed
$s_i \approx 0$: on a border
$s_i < 0$: probably in the wrong cluster

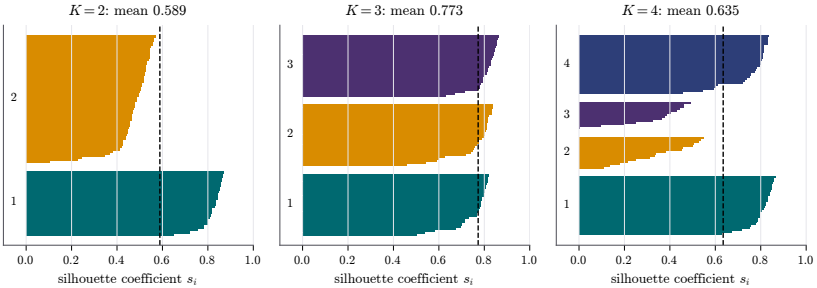
Choosing K with the silhouette

Silhouette score = mean of s_i over all points. Pick the K with the largest score.



Silhouette plot: which cluster is wrong

One bar per point, sorted within each cluster. Dashed line: silhouette score.



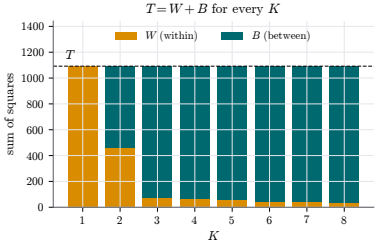
3. Calinski–Harabasz

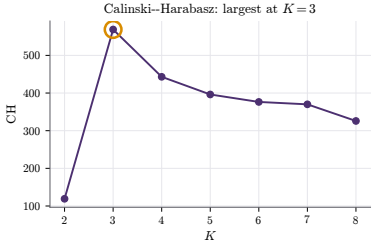
$$\mathrm{CH}(K) = \frac{B/(K-1)}{W/(n-K)}$$

$K = 3$: $507.2/0.892 = 568.8$
$K = 4$: $341.9/0.772 = 443.1$

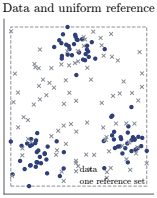
Pick the largest: $K = 3$.

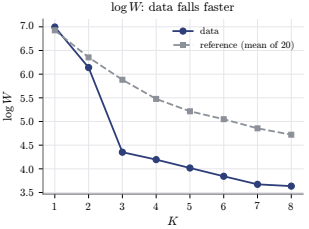
B: spread between cluster means
W: spread inside clusters
($T = W + B$)

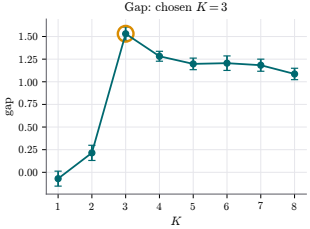




4. Gap statistic







$$\mathrm{Gap}(K) = \overline{\log W_K^*} - \log W_K$$

W_K^*: the same W_K computed on random uniform data in the same box (20 copies).

Pick the K where the data beat random data by the most: here $K = 3$.

5. Domain knowledge

- 3 marketing budgets $\Rightarrow K = 3$
- 4 T-shirt sizes $\Rightarrow K = 4$
- 2 machine types $\Rightarrow K = 2$
- 16-colour image $\Rightarrow K = 16$

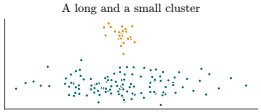
Insight

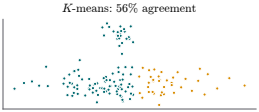
Often the application decides K.

Limitations of K-means

Limitation	Where it bites
K must be chosen	data with no clear groups
Result depends on the start	a random start split a group
Depends on feature units	hours vs megabytes
Outliers pull centroids	one extreme run drags a mean
Round, similar clusters; straight boundaries	long clusters, half-moons
Numbers and Euclidean distance only	categories, e.g. GPU type

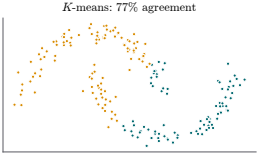
Where K-means fails: shape





Long + small cluster:
56% agreement





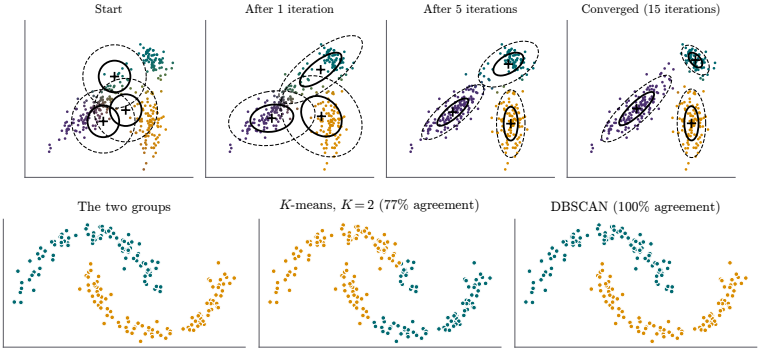
Half-moons:
77% agreement

Boundaries are straight lines.

What handles these cases

Gaussian mixtures
each cluster an ellipse

DBSCAN
clusters as dense chains



Live demonstration

`unsupervised_demo.ipynb`, Part A

1. Pizza outlets, round by round
2. Random vs K-means++ starts
3. Elbow and silhouette
4. Round groups, then half-moons

Review questions

1. Why can the update step not increase J?
2. Points $1, 2, 9$, start at 1 and 2: final J?
3. Why does the smallest J never rise with K?
4. Why random, not farthest, in K-means++?
5. Why divide by $K - 1$ and $n - K$ in CH?

Handbook Chapter 2, Problems 2.1–2.15.